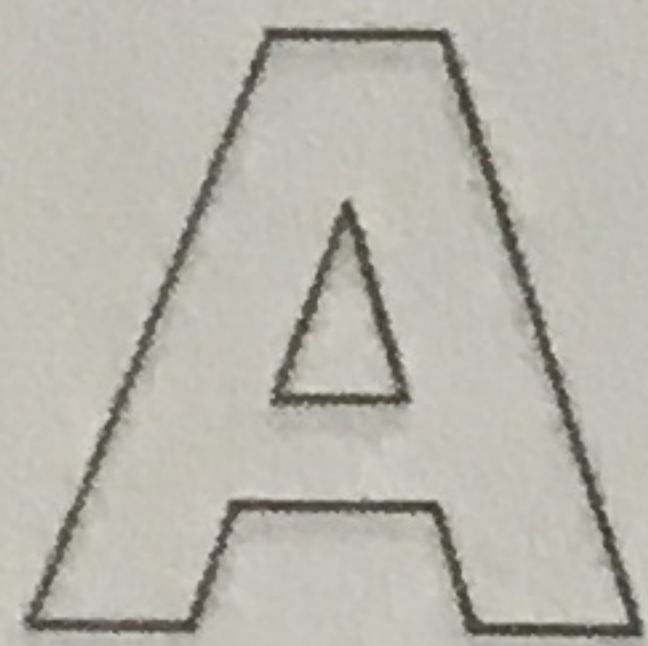




Pe prima pagină de examen scrieți în colțul dreapta sus
Numele și prenumele
Grupa
Varianta de subiect

Examen POO – 2 februarie 2019

1p din oficiu

1. (0,5p) Definiți conceptul de **polimorfism** în Programarea Orientată pe Obiecte. Dați un exemplu simplu în JAVA.
2. (0,5p) Dați un exemplu în JAVA de o clasă cu o **variabilă instanță** care poate fi vizibilă în orice altă clasă din același pachet cu clasa respectivă, dar nu poate fi vizibilă în nicio altă clasă din afara pachetului.
3. (0,5p) Definiți conceptul de **ambiguitate la supraîncărcarea metodelor** în Programarea Orientată pe Obiecte. Dați un exemplu simplu în JAVA.

4. (0,5p) Explicați ce se întâmplă în urma compilării și execuției următorului program JAVA:

```
public abstract class A {  
    public static String t="Examen ";  
    public void method() {  
        t="Iarna ";  
    }  
}
```

```
public class B {  
    public static void main(String args[]) {  
        System.out.println(A.t);  
    }  
}
```

5. (0,5p) Explicați ce se întâmplă în urma compilării și execuției următorului program JAVA:

```
public class A {  
    private int x;  
    public A (int y) {  
        x = y + 10;  
    }  
}
```

```
public class B extends A {  
    private int x;  
    public B (int y) {  
        x = y + 20;  
    }  
    public static void main(String args[]) {  
        B obj = new B(50);  
        System.out.println(obj.x);  
    }  
}
```

6. (0,5p) Definiți conceptul de **subinterfață** și dați un exemplu simplu de scriere a uneia în JAVA.

7. (0,5p) Dați un exemplu în JAVA de o clasă cu o metodă non-statică care aruncă o **excepție verificată** pentru un anumit obiect și care ulterior este interceptată într-o altă metodă statică a clasei.
8. (0,5p) Dați un exemplu în JAVA de folosire a unei **colecții** în care să rețineți o mulțime de numere întregi, astfel încât să nu existe duplicate, iar tipărirea elementelor colecției, în ordinea crescătoare a valorilor, să se facă oricând cât mai eficient posibil, chiar dacă inserarea/căutarea elementelor nu sunt cele mai eficient posibile operații.
9. (0,5p) Definiți conceptul de **jurnalizare** și dați un exemplu simplu în JAVA.

10. (1,5p) Ce se afișează la ieșirea standard în urma execuției următorului program JAVA:

<pre> class A { static String y = "POO "; int x = 20; public A(int x) { y += "Sesiune "; x = 25; } boolean equals(A obj) { return x==obj.x; } public int method() { return 2*x; } } class B extends A { static { y += "Examen "; } int x = 30; public B(int x) { super(x); y += "Iarna "; } public int method() { return 3*x; } } </pre>	<pre> public class C { public static void main (String[] args) { System.out.println(A.y=="POO "); B b = new B(100); System.out.println(A.y); A a = new B(100); System.out.println(a==b); System.out.println(a.equals(b)); System.out.println(a.x); System.out.println(b.x); System.out.println(a.method()); System.out.println(b.method()); System.out.println(a.getClass().getName()); A obj = new A(100); System.out.println(obj instanceof B); } } </pre>
---	--

11. (3p) O parcare de autovehicule este de formă dreptunghiulară (matriceală), având un anumit număr de linii și coloane, determinând astfel un număr de locuri de parcare disponibile egal cu produsul dintre numărul de linii și coloane. Pe un anumit loc de parcare poate fi parcat un singur autovehicul, acesta fiind fie autoturism fie motocicletă. Pentru orice autovehicul se reține marca și numărul de cai putere. Pentru un autoturism se reține în plus valoarea kilometrajului iar pentru o motocicletă faptul dacă are sau nu ataș.

Respectând paradigma programării orientată pe obiecte, identificați entitățile, relațiile dintre acestea, desenați diagrama de clase UML detaliată și implementați în JAVA un program care să rezolve următoarele cerințe:

- Crearea de parcări prin citirea caracteristicilor de la intrarea standard
- Crearea de autoturisme și motociclete
- Parcarea unui autovehicul pe un anumit loc de parcare – se va utiliza un tablou bidimensional pentru reținerea locurilor de parcare în acel spațiu dreptunghiular (matriceal) al parcării. În cazul în care se încearcă parcarea unui autovehicul la o anumită poziție (rând, coloană) unde există deja un autovehicul parcat, se va afișa automat un mesaj de eroare.
- Eliberarea unui anumit loc de parcare (rând, coloană)
- Determinarea numărului total de autoturisme de pe un anumit rând al parcării
- Determinarea numărului total de motociclete fără ataș din întreaga parcare
- Afișarea tuturor detaliilor despre toate autovehiculele aflate în parcare în ordinea descrescătoare a numărului de cai putere
- Scrieți o metodă *main* în care creați mai întâi o parcare, apoi executați toate cerințele specificate mai sus astfel încât să utilizați fiecare funcționalitate cel puțin o dată